

# SE LAB 3 \_ Kamal Kanth N(PES1UG24AM434)

## Problem Statement 60 — Podcast Guest Scheduling & Outline Builder

### 1. Architecture Selection

For the Podcast Guest Scheduling & Outline Builder system we chose Layered Architecture.

The system was compared with the Layered, Micro services and Client-Server styles. Micro services was ruled out due to the tight sequential nature of the workflows (booking, outline review, production-sheet generation) and their sharing of the same core data. Splitting them into independently deployed services would introduce network latency and operational overhead that the project doesn't need at this scale, and would work against the tight sub-1-second production-sheet requirement (NFR-001). A simple two tier Client-Server style was also thrown out as it doesn't cleanly separate the guest/host facing UI from the booking and outline business rules, making it harder to reuse the same validation logic across the guest booking flow and the host review flow.

### 2. Two Scenario-Related Reasons

- Shared business logic across two user roles: Both the podcast guest (booking, outline submission) and the show host (slot management, outline review, production-sheet generation) drive the same underlying rules — e.g. a slot must be validated as unblocked (FR-002) before it can be booked (FR-001), and every approved outline must be present with its timestamp and bio link before the production sheet can be generated (FR-005). A distinct business layer means this logic is written once and enforced consistently for every entry point, instead of being duplicated in the UI or scattered across independent services.
- Simple, sequential workflow with a single evolving dataset: Interview slots, topic outlines, and the production sheet are really one connected dataset that gets refined stage by stage. A layered design lets every component in the business layer talk to one persistence layer, which keeps the data consistent without the distributed-transaction problems that micro services would introduce for this same data.

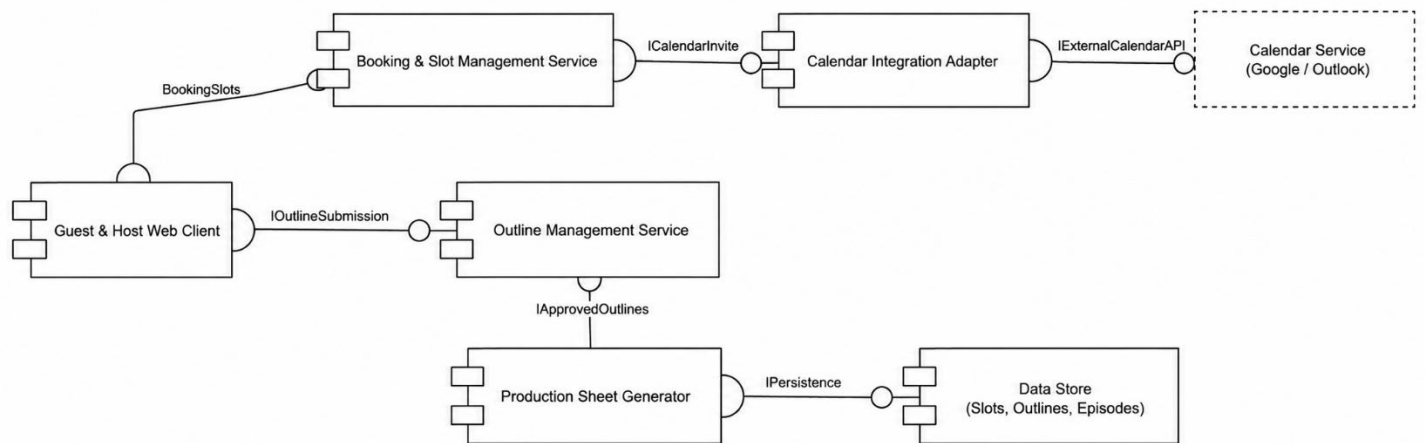
### 3. Security Advantage

Placing scheduling and outline rules in a dedicated business layer means the presentation layer (the guest/host web client) never talks to the data layer directly — every request must pass through business-layer checks such as "is this slot still open" (FR-001 acceptance criteria) or "is this date blocked by the host" (FR-002). This closes off the failure mode where a guest could otherwise manipulate a client-side request to book a blocked date or attach an outline to an episode they don't have access to: the enforcement point is centralised in one layer rather than trusted to the client.

### 4. Performance Benefit

NFR-001 requires the production sheet to be compiled into a formatted PDF in under 1 second. Because the Production Sheet Generator sits in the business layer with a direct, in-process interface to the persistence layer it can pull all approved outlines for an episode in a single data-layer round trip rather than making repeated calls across service or network boundaries, as a micro services split would require. This keeps the compile step fast and predictable even under the host's typical "minutes before recording" usage pattern.

## 5. Components & Interfaces



Six components :

Guest & Host Web Client (presentation),

Booking & Slot Management Service,

Outline Management Service,

Production Sheet Generator,

Calendar Integration Adapter (business layer),

shared Data Store (data layer),

external Calendar Service.

Ball-and-socket notation is used throughout the diagram, with six labelled interfaces:

IBookingSlots,

IOutlineSubmission,

ICalendarInvite,

IExternalCalendarAPI,

IApprovedOutlines,

IPersistence interface

The Booking and Outline services persist data through the same Data Store interface as the Production Sheet Generator, so only one representative connection is drawn to keep the diagram readable.